



UNIVERSITÀ
DEGLI STUDI
DI UDINE
HIC SUNT FUTURA

DEPARTMENT OF MATHEMATICS, COMPUTER SCIENCE AND PHYSICS

BACHELOR THESIS IN
INFORMATICA

Develop an intelligent conversational systems for personalized cycling trip planning

CANDIDATE

Riccardo Gottardi

SUPERVISOR

Prof. Giuseppe Serra

CO-SUPERVISOR

Marco Passon

Academic Year 2024-2025

INSTITUTE CONTACTS

Dipartimento di Scienze Matematiche, Informatiche e Fisiche

Università degli Studi di Udine

Via delle Scienze, 206

33100 Udine — Italia

+39 0432 558400

<https://www.dmif.uniud.it/>

Abstract

This thesis presents the design and implementation of a conversational system for personalised cycling trip planning, addressing the gaps in the current cycle tourism market. Existing cycle route planning tools lack comprehensive user guidance, metrics and preferences. The research objectives include the development of a conversational AI system that can collect trip related information through natural language interaction, generate a cycling-specific route, and provide personalized recommendations.

The implementation combines large language models (LLMs) with conversational AI techniques, specifically implementing the ReAct (Reason and Act) framework for structured reasoning and action. The system architecture was realized using Pydantic-API¹ as the agentic framework, integrating external services Nominatim², Brouter³ and Overpass API⁴, respectively, for geocoding, cycling-specific routing and point of interest (POI) recommendations. Lastly, data integrity and validation are realized through custom data-structures (`TripDescriptor` and `UserDescriptor`).

An experimental validation was conducted on the developed system through the systematic testing of 30 scenarios of varying complexity, comparing the performance of a system with an agent instructed on a baseline prompt (a structured prompt) with a prompt enhanced with the ReAct approach. As the result of the experimentation, the system powered by the enhanced prompt has achieved an 80% success rate, while the baseline a 67%, representing a 27% relative improvement for the enhanced prompt. Additionally, the ReAct approach helped in reducing the agent hallucinations from a 17% to a 13%, demonstrating a higher reliability in the information processing task.

The results validate the feasibility of applying conversational AI to cycle tourism planning, and successfully demonstrate the core functionality of information gathering and route generation. The prototypes represent an advanced proof-of-concept, transitioning toward a minimum viable product (MVP).

¹Pydantic-AI: <https://ai.pydantic.dev/>

²Nominatim: <https://nominatim.org/>

³Brouter: <https://brouter.de/brouter/>

⁴Overpass API: <https://overpass-api.de/>

Contents

Contents	v
1 Introduction	1
1.1 Context of cycle tourism and its digitalization	1
1.2 Project motivations and objectives	1
1.2.1 Market analysis	1
1.2.2 Academic objectives	2
1.2.3 Writer objectives	2
1.3 Thesis structure	2
2 Background theory and state-of-the-art	5
2.1 NLP foundation	5
2.2 LLM capabilities	6
2.3 AI agents fundamentals	7
2.4 Prompting	7
2.5 Conversational AI basics	8
2.6 Geospatial data management	8
3 Requirements analysis and design	11
3.1 Application domain	11
3.1.1 Definition of cycle tourism	11
3.1.2 Domain characteristics and requirements	11
3.1.3 System scope within the domain	12
3.2 Analysis of the requirements	13
3.2.1 Functional requirements	13
3.2.2 Non-functional requirements	14
3.3 Evolution of the system architecture	14
3.3.1 First iteration	14
3.3.2 Second iteration	15
3.4 Technological choices	16
4 System implementation	19
4.1 Final architecture	19
4.1.1 Component architecture and dependencies	19
4.2 Data structures	20
4.2.1 <code>TripDescriptor</code> implementation	21
4.2.2 <code>UserDescriptor</code> implementation	23
4.2.3 Tool structure	24
4.3 Conversational agents	24
4.3.1 Framework integration	24
4.3.2 Configuration management	24
4.3.3 Tool integration	25
4.4 Prompt engineering and desing	25

4.4.1	ReAct approach implementation	26
4.4.2	Workflow deligation	27
4.4.3	Error handling	27
4.5	Integration with external services	27
4.5.1	Geocoding services	28
4.5.2	Routing services	28
4.5.3	Recommendation service	29
5	Testing and validation	31
5.1	Testing methodologies	31
5.1.1	Black-box component testing	31
5.1.2	Bottom-up integration testing	32
5.1.3	Approach trade-offs	32
5.2	Agent behaviour testing	32
5.2.1	Framework integration	32
5.2.2	Monitoring capabilities	33
5.2.3	Traces analysis methodology	33
5.3	Experimental validation design	34
5.3.1	Experimental objectives	34
5.3.2	Experimental subject	34
5.3.3	Test scenarios development	35
5.4	Agent experimental evaluation	35
5.4.1	Evaluation framework architecture	35
5.4.2	Binary classification methodology	35
5.5	Experimental results	36
5.5.1	Overall performance comparison	37
5.5.2	Information quality analysis	37
5.5.3	Message complexity analysis	37
5.6	Conclusions	39
6	Discussion and limitations	41
6.1	Evaluation of the developed prototype	41
6.1.1	Technical achievements	41
6.1.2	Prototype maturity assessment	41
6.2	Current limitations	42
6.2.1	Technical limitation	42
6.2.2	Functional limitation	42
6.3	Further development	43
7	Conclusion	45
	Bibliography	47

Introduction

1.1 Context of cycle tourism and its digitalization

In Italy, between the years 2023 and 2024, cycle tourism has produced as revenue nearly 9.8 billion euros, with around 89 million tourist visiting the country during their cycling journey. Those numbers represent a growth of 54% on the number of tourists choosing Italy as part of their cycling vacation from the preceding years [1]. This growth was not only limited to Italy, the same trend have been observed throughout Europe, where cycle tourism have generated approximately 44 billion euros [2].

The tourism following the evolution of society, has undergone a profound transformation over the past decades. Indeed the tourism industry has come from tourism 1.0, with basic digital presence, after the advent of the Web 1.0 with tourism industries that were using it as a one way communication medium for advertisement to the tourism 5.0, where technology belonging to the Web 5.0 (like the metaverse) were used in it. However, for the sake of this thesis the interest type of tourism is the 4.0, that has seen the integration of technologies like Artificial Intelligence (AI) and Big Data [3].

Current market analysis shows that 63.1% of the cycle tourists rely on the internet to plan their cycling experience. However, as it will be explained in Section 1.2.1 many of those tools offer a fragmented experience, lacking integration between the different activities involved in cycling trip planning and user guidance [1].

1.2 Project motivations and objectives

1.2.1 Market analysis

The majority of existing digital platforms for cycling tourism are single-purpose software focused on specific functionalities. However, a few solutions (such as Strava¹, Komoot² and Bikemap³) address a broader range of features. They usually allow route calculation based on user terrain preferences and fitness level, break routes down into smaller steps based on the number of days users want to spend on their trip, and include community functionality to share session recordings, inspire other users, and

¹Strava: <https://www.strava.com/>

²Komoot: <https://www.komoot.com/>

³Bikemap: <https://www.bikemap.net/>

provide recommendations.

Although the route planning features are pretty solid for most of the available platforms and allow users to specify their preferred terrain type, they are still basic when it comes to route segmentation. They usually only take into account the number of days that the trip will last, rather than the user's capabilities. These capabilities are used to calculate the route, but it is unclear what they represent since they do not map to precise performance measurements (for example, words like 'fit', 'advanced' and 'beginner' are used to describe user capabilities). Furthermore, they lack user guidance. Route refinement is completely up to the user. It is possible to add points of interest (POIs), but this has to be done manually. Indeed, it is up to the user to identify the most useful and interesting POIs and add them to the trip.

To sum up, the cycling tourism market lacks comprehensive software that can support users in planning cycling activities, from basic route generation to refinement and segmentation. Recent advancements in large language models (LLMs) and conversational AI have led to their integration into systems for customer support or general chatbots, such as ChatGPT⁴. In the writer's opinion, these technologies could be applied to the cycling tourism market, from which it would benefit.

1.2.2 Academic objectives

The Cycling Trip Agency project aims to develop an intelligent conversational system that can plan a fully personalised cycling trip based on the user's preferences and capabilities. Users will only need to share their ideas and the system will be responsible for planning the trip, suggesting potential routes and points of interest, and breaking it down into multiple steps. The user only needs to select their preferred option from those proposed by the system, rather than actively researching a solution themselves.

1.2.3 Writer objectives

The aim of this thesis is to design the system architecture and provide a working prototype that implements the first agent with which the user will interact. This agent will be responsible for gathering the information needed to plan a basic trip and creating the first route.

1.3 Thesis structure

This thesis is divided into seven chapters that address the various stages of developing the prototype. These range from the theoretical background to the design, implementation and validation stages.

Chapter 2 presents the foundational knowledge and the state of the art of the tools that will be used within the project: These include NLP (Natural Language Processing), LLMs (Large Language Models), AI Agents, Conversational AI, and Geospatial Data Management.

Chapter 3 defines the application domain, analyses the system requirements, and presents the system architecture, explaining the technological choices.

Chapter 4 presents the final system architecture and explains how the system was implemented. This includes everything from data structures to conversational agents and agent prompting.

⁴ChatGPT: <https://chatgpt.com/overview>

Chapter 5 addresses system and component testing and system validation. It outlines the reasons why certain methodologies were adopted for testing and validation.

Chapter 6 evaluates the developed prototype, highlighting its strengths and weaknesses, and proposes future developments.

Chapter 7 draws conclusions. It synthesizes the research results and outlines future research directions for the project.

Background theory and state-of-the-art

This chapter introduces the technologies that power the project's tools and provides a brief explanation of how they work. In Sections 2.2 and 2.1, are presented respectively Large Language Models (LLMs) and Natural Language Processing (NLP), the foundational technologies for AI agents, the core technology of the project, presented in Section 2.3, while in Section 2.4, prompting, the method through which LLMs and AI agents are instructed, is presented. Since the project aims to develop a conversational system, Section 2.5. Finally, as the system will integrate geospatial services for route calculation and geocoding, Section 2.6 presents the fundamentals of geospatial data management.

2.1 NLP foundation

Natural language processing (NLP) is a branch of computer science that uses machine learning to enable computers to understand and use natural language. It makes it easier for humans to interact with machines using the modality they use every day. Its applications benefit many fields, from automating repetitive tasks such as answering customer support queries, classifying documents and summarising them, to enhancing search by understanding the user's intention behind the query and providing relevant content, even when the query is malformed.

When it comes to implementing NLP, there are different approaches that can be taken:

- **Rule-based:** NLP provides a basic form of NLP and natural language generation (NLG), offering answers to specific prompts only. It is made up of a chain of pre-programmed conditional statements.
- **Statistical:** NLP extracts and classifies text and speech data, assigning a statistical likelihood to each possible meaning. This approach introduced the foundational technique of mapping words and grammatical rules to vector representations, enabling them to be modelled using mathematical methods [4].
- **Deep learning:** NLP can be viewed as an evolution of statistical NLP, incorporating neural network (NN) models.

NLP involves combining different techniques.

- **Text preprocessing:** Involves transforming received text into data that can be analysed and processed further. This is usually done in several phases:
 - *Tokenisation:* The text is divided into smaller units, such as words or phrases.
 - *Lowercasing:* Is then used to standardise the text, ensuring that tokens such as 'dog' and 'Dog' appear the same.
 - *Stop word removal:* Involves removing words that do not add meaning, such as 'is' and 'the'.
 - *Stemming:* Reduces words to their root form (e.g. 'cycling' becomes 'cycle').
 - *Text cleaning:* Is an optional phase where special characters are removed.
- **Feature extraction:** Tokens are transformed from their textual representation into a numerical one that a machine can understand. Techniques such as the 'bag of words' approach or more advanced methods such as word embedding are applied to achieve this.
- **Text analysis:** Involves extracting information from the transformed text. This involves different computational techniques, each responsible for extracting a specific piece of information, such as sentiment analysis and topic modelling.
- **Model training:** Once the data has been processed, it can be used to train machine learning models to identify patterns and relationships within the data.

2.2 LLM capabilities

Concerning LLMs, the state of the art is represented by Transformer-based models, which can be divided into two parts: an encoder and a decoder. The encoder is responsible for understanding text, while the decoder is responsible for generating text [6]. Based on this, three types of Transformer can be defined:

- **Encoder-only:** Suitable for classification tasks such as sentiment analysis.
- **Decoder-only:** Those models are suitable for language and content generation tasks, such as writing stories or blogs.
- **Encoder-decoder:** This kind of models combine the capabilities of the above models. It can achieve both classification and content generation, as well as tasks such as translation and summarisation.

Behind the success of the transformer based models there are two key innovations:

- **Positional encoding:** This is a process in which the order in which a sequence appears in the input is embedded into it. This technique enables different sequences to be processed non-sequentially, allowing complex problems to be decomposed into smaller ones that can be processed simultaneously [6].
- **Self-attention:** It is a mechanism through which LLMs assign a weight to the different parts of the input based on their importance in the context of the rest of the input. This enables LLMs to focus only on the relevant parts [6].

2.3 AI agents fundamentals

An agent, specifically, a generative AI agent, can be defined as an application that aims to reach a goal by understanding the world and acting upon it using the tools at its disposal [7]. They comprise three main components:

- **Model:** This is a language model (LM) which can be large or small and acts as the agent's brain [7].
- **Orchestration layer:** Describes the process by which the agent takes input, performs internal reasoning, and uses that reasoning to infer its next action or decision [7].
- **Tools:** Serve the role of enabling the agent to interact with external data sources and services [7].

Another aspect to consider when defining AI agents is the architecture of the agentic system. The choice of architecture is driven by the complexity of the task to be addressed and how it is to be addressed, i.e. the desired workflow. It is usually common sense to start with the simplest possible architecture (even considering whether an agentic system is needed for the task), and only expand it if necessary. There are several architectures; below, we will present some of them:

- **Augmented LLM:** A workflow involving a single LLM with access to different tools to satisfy user queries [8].
- **Prompt chaining:** Is a multi-agent architecture suitable for tasks consisting of smaller steps that require sequential execution. Each of these steps is handled by a separate agent and all of them are executed sequentially [8].
- **Routing:** Is a multi-agent architecture with an entry point in the form of an agent (the router), whose goal is to send the user query to the most suitable agent in the system for the task [8].
- **Orchestrator-workers:** A multi-agent workflow that requires an orchestrator agent to break down the query into smaller tasks and delegate each one to the appropriate agent [8].

2.4 Prompting

Prompting is a fundamental technique through which agents, and more generally LLMs, can act. There are two types:

- **User prompts:** These are the messages that users send to agents. These describe the activity that the model or agent has to perform and the user's expectations [6].
- **System prompts:** Are the messages that shape the behaviour of the agent or model. Through them, many things are specified, such as the tasks to be performed, how the agent or model should behave during execution, and how the agent should approach the assigned task. As they instruct the agent or model on a specific task for which it may not have been explicitly trained, system prompts play a key role in how an agent or model performs on those specific tasks [6].

Several prompting strategies already exist, and the ones that will be used in the project, along with some of the most important ones, will be shown below.

- **Reason and act (ReAct):** This synergises acting and reasoning abilities, allowing humans to learn and make decisions. This paradigm provides an interleaved presence of reasoning traces and task-specific actions. The reasoning traces serve for action planning and exception handling, whereas the action steps serve for information retrieval [9].
- **Few-shot:** Is a technique that enables in-context learning by providing the LLM with examples of execution. The goal is to show the LLM the desired execution flow and output based on an input so that it will follow the example in future executions; these examples act as guidelines for behaviour.
- **Chain of thought (CoT):** Is a technique that addresses complex reasoning by breaking it down into smaller steps. The concept is similar to the 'divide et impera' principle, whereby a complex problem is first divided into smaller, simpler problems. The only difference is that, in the case of CoT, it is the reasoning that is broken down [10].

2.5 Conversational AI basics

Conversational AI is a form of artificial intelligence that uses natural language processing (NLP) to simulate human conversations. Such systems are trained using large data sets, including text and audio sources, to learn how to comprehend and use human language. They then use this knowledge to interact naturally with humans and use these interactions to further improve their response capabilities [11]. These systems are mainly made up of two components:

- **Machine learning:** Whose algorithms are key to improving based on experience [12].
- **NLP:** Represents the way the conversational system is able to comprehend user input.

Conversational AI has proven useful for automating tasks currently handled by humans, reducing costs and errors while increasing user satisfaction and engagement [11]. Examples of its application in task automation include chatbots for customer support, which can answer user queries 24/7, and the Internet of Things (IoT), where smart assistants like Alexa¹ and Google Home² use automated speech recognition to interact with users.

2.6 Geospatial data management

Georouting technology has evolved beyond traditional shortest-path algorithms, particularly for specialised applications such as cycling tourism. Nowadays, advanced algorithms and AI-driven personalisation are integrated to address the different requirements of bicycle routing, which differ from those of automotive navigation since more factors need to be taken into account, such as surface type and elevation constraints.

A state-of-the-art solution encompasses three technological components:

¹Alexa: <https://www.amazon.it/Echo-e-Alexa/b?ie=UTF8&node=15619933031>

²Google Home: https://home.google.com/intl/it_it/welcome/

- **Routing:** Algorithms enhanced with preprocessing techniques such as Contraction Hierarchies, which are useful for drastically reducing query times on continental-scale networks.
- **Open-source:** Geospatial services that include different services for different purposes, such as a geocoding layer for converting human-readable addresses to coordinates (e.g. Nominatim³), or a routing engine (e.g. Brouter⁴).
- **AI-powered:** Personalisation is carried out by graph neural networks (GNNs) that model spatial dependencies and take into account other factors, such as weather conditions and fitness level, allowing for a tailored cycling tourism application.

³Nominatim: <https://nominatim.org/>

⁴Brouter: <https://brouter.de/brouter/>

3

Requirements analysis and design

This chapter defines the application domain of the cycling trip planning system, and provides the foundation for its development. The chapter begins by defining cycle tourism and analysing its unique characteristics that differentiate it from other forms of travelling (Section 3.1). Section 3.2 provides a comprehensive analysis of both functional and non-functional requirements, derived by intersecting the gaps identified in Section 1.2.1 and the needs underlined in Section 3.1, in accordance with the aims described in Section 1.2.2. The evolution of the system architecture is traced in Section 3.3, from the initial concepts to the final multi-agent design, showing the different iterations that the system has undergone. Finally, Section 3.4 justify the technological choices made during the development.

3.1 Application domain

3.1.1 Definition of cycle tourism

Cycle tourism represents a specialised form of sustainable tourism, where cycling serves as both primary means of transportation and recreational activity. The definition of cycle tourism encompasses a variety of activities, ranging from holidays and short breaks that incorporate cycling as part of the overall experience, to trips where cycling is the primary focus or a significant component of the itinerary [13]. This definition includes various forms of cycling-based travel, from single-day trips to multi-day touring experiences.

What distinguishes cycle tourism from the other types of travelling is its need for feasibility driven route planning, based on the physical performance of the cyclist, that also takes into account factors such as the terrain type. Unlike conventional tourism, where transportation is just a way to reach the destination, in cycle tourism, it is the main part of the tourist journey, requiring, indeed, careful planning.

3.1.2 Domain characteristics and requirements

As mentioned in the previous section (Section 3.1.1), cycle tourism differs from the classical form of tourism by the way in which planning is approached, since the factors considered in it:

- **Physical performance metrics:** Represent the more distinguishing aspect of cycle tourism

planning, since, by its nature, cycling imposes a direct physical demand that can vary among the users. The system has to take into account the performance metrics of the user, such as daily distance capability and elevation gain tolerance, since they directly influence the route segmentation, requiring the system to model the user capabilities as quantitative constraints rather than generic skill categories.

- **Route-specific requirements:** Different bike types allow for different terrain types, a road bike requires paved surfaces, a gravel bike can handle unpaved surfaces with moderate off-road segments, and lastly, a mountain bike can handle off-road sections, but may be suboptimal for long-distance routing.
- **User diversity:** Cycle tourists range from casual day-trip cyclists to experienced ultra-distance riders, each with different preferences for POIs, accommodation types and risk tolerance. Therefore, the system must accommodate this diversity through preference modelling that encompasses individual interests, such as amenities, repair shop or pharmacy, to address different kinds of emergency and more.

3.1.3 System scope within the domain

The Cycling Trip Agency system operates with the goal of addressing specific gaps in the current cycle tourism planning ecosystem. This section delineates the system coverage area and value that it aims to represent.

- **System coverage:** Encompasses end-to-end conversational trip planning, from an initial idea to route generation, eventual POIs integration and segmentation. The system focuses on the planning phase of cycle tourism, addressing information gathering, route calculation, and personalisation processes through natural language interaction. The user engages with the system, expressing their ideas about the trip in a conversational form, without the need for manual research.
- **Boundaries of the existing solution:** Despite general mapping tools, like Google Maps, offer routing capabilities, they lack cycling-specific routing metrics. Specialized tools like Strava¹ and Komoot² (one more focused on the community side of cycle tourism, with session sharing and other features), if in one hand they do offer cycling-centric route planning, on the other hand they require the user to manually feed the system and actively search for POIs, therefore lacking in guidance. On top of that, individual performance metrics are ignored when segmenting and creating the route, possibly resulting in unfeasible trips.
- **Value:** The system aims to eliminate the friction during the planning phase of a cycle tourist experience, providing guidance during the planning phase to the user, interpreting its requests and proposing solutions, building together the user desired trip, with a low demand on the user.
- **System limitations and goals:** Are acknowledged to maintain a realistic scope. The system does not address real-time navigation, accommodation booking, or post-trip services. Indeed, the

¹Strava: <https://www.strava.com/>

²Komoot: <https://www.komoot.com/>

system addresses the pre-trip planning phase, with a specific focus on the information gathering and route planning, addressing a really basic recommendation system, more as a proof of concept.

3.2 Analysis of the requirements

Requirements analysis translates domain characteristics and system scope into specific functional and non-functional specifications that guide the system design and implementation phases. This analysis is based on the market gaps identified in Section 1.2.1 and domain understanding established in Section 3.1.

3.2.1 Functional requirements

Functional requirements define the main capabilities that the system must provide to address the needs underlined in the section above. Following are the identified requirements:

- **Information gathering requirements** Address the ability of the system to extract parameters needed to plan a trip through natural language conversation. The system has to collect the mandatory information to plan the trip: bike type, locations the user wants to go through during the trip, at least a starting and ending point, and the user performance metrics (kilometres per day and elevation gain). Other information can be extracted from the conversation, like the desired trip duration and preferences about the POIs, but since they are not mandatory, the system will not ask for them before proceeding with the route planning. Examples of natural language input that the system must handle include:
 - “I want to plan an off-road cycling trip that goes from Udine to Pordenone”
 - “I have a road bike and what to plan a trip from Padova to Trieste. Currently I can handle 86km per day”
 - “I can have 3 days for a trip that goes from Polcenigo to Aquileia, do you think I can do it ? Can you help me plan it ?”

The system must extract structured information from the messages above, identifying locations to go through (Udine to Pordenone, Polcenigo to Aquileia), the trip duration (3 days), bike type (road, off-road, so mountain bike) and the kilometres per day (86km).

- **Trip planning requirements:** The system, based on the information extracted from the conversation with the user, must be capable of planning a cycling-specific route among the extracted locations, taking into account terrain type, determined by the type of bike that will be used for the trip. In a second moment, it should break down the route into manageable steps, based on the performance metrics already extracted, since they are mandatory information.
- **Personalization requirements:** The system should be able to provide POI suggestions based on the user preferences expressed during the dialogue between them. If the user expresses the will of integrating those POIs to the trip, the system should do so and proceed with the planning of a new route based on the updated trip information.

3.2.2 Non-functional requirements

Non-functional requirements define quality attributes and operational constraints the system has to undergo. The requirements have to describe measurable criteria that complement the functional capabilities discussed in Section 3.2.1. Following is the established non-functional requirements:

- **Usability:** A user who has not interacted with and seen the system before has to be able to plan a trip using the system without support other than the system itself.

3.3 Evolution of the system architecture

In order to design the system architecture, the required activities to accomplish them have to be listed first. Then, they can be grouped and each group assigned as the responsibility for a specific part of the system.

Activity involved:

- As the first activity, some information has to be collected to plan the trip, so the system and the user should interact to establish the mandatory knowledge to plan the trip.
- Once the essential information to plan the trip has been collected, the route can be planned and presented to the user (eventually refining it).
- After the user is satisfied with the first iteration of the route, it can be further modified, adding POIs based on its preferences, therefore its preferences should be extracted through conversation.
- Once added, the optional POIs to the locations to go through during the trip, it is time to plan the route again.
- Finally, the route can be segmented. If the information to base the segmentation upon were not already collected, they should be through conversation.

3.3.1 First iteration

From the activities listed above, three are the main phases involved in the trip planning: base route planning, personalisation with POIs, and segmentation. To address those phases, the following agents were designed:

- **Information collector:** Each activity requires specific information to be carried out; those information are collected during a defined phase in which the system and the user interact to state them and make them part of their common knowledge. By the nature of the system, each time it interacts with the user, it is because it needs an information, that could be a key information to plan the trip or to segment the route, or a confirmation that allow the system to go forward. It is for that reason that a specialized agent was outlined, responsible only for information gathering. It is the only point of the system charged of interacting with the user, the other agents should have been specialized and focused on the technical part of the task they were charged with, delegating the interaction with the user, and all the aspects it encompasses (how to interact, which information to present and which not, and other aspects), to the information gathering agent.

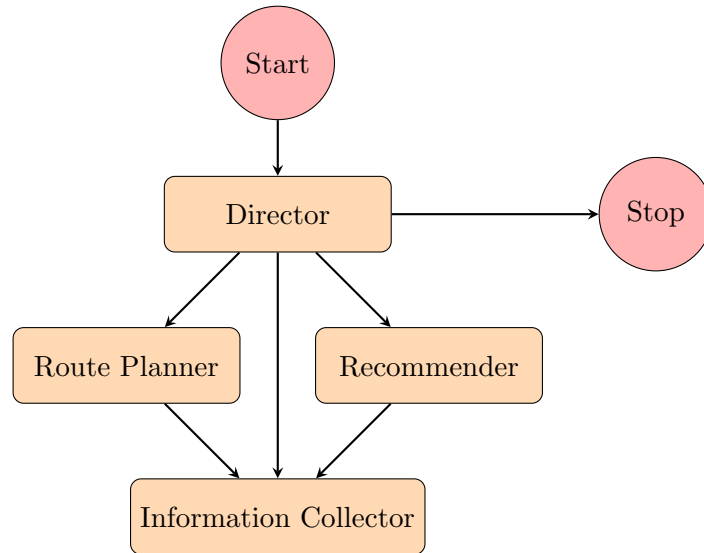


Figure 3.1: Initial architecture of the system.

- **Route planner:** Responsible for the route planning, refinement and segmentation.
- **Recommender:** Responsible for the optional part of POIs discovery and integration into the places to go through the trip.
- **Director:** Its goal is to coordinate the transition point between each different phase. Once an agent has finished a phase, the Director has the responsibility of verifying that there were all the conditions necessary to proceed with the next one, and proceed by delegating the execution of the remaining phases.

The architecture designed so far presents some critical aspects:

- **Information collector complexity:** The problem with the logic behind it is that, when called, the agent not only needed the information to gather, but also the context in which they were requested. The context parsing required sometimes to share a long description of what the collector should have had to ask and why, actually doing more work than directly interacting with the user.
- **Director existence:** It has not made an activity more complex than it actually is, like in the case of the information collector, but its existence is pointless. Indeed, as explained above, it is responsible for the coordination of the other agents, but since the workflow for planning to plan a trip, outlined above, is linear, there is not a real need for an orchestrator.

3.3.2 Second iteration

Taking into account the critical aspects of the first architectural design, the two critical agents were removed, maintaining the Route Planner and the Recommender. Now, each agent was not only responsible for carrying out the task they were assigned to, but also responsible for the gathering of the information needed to perform the task they are responsible for.

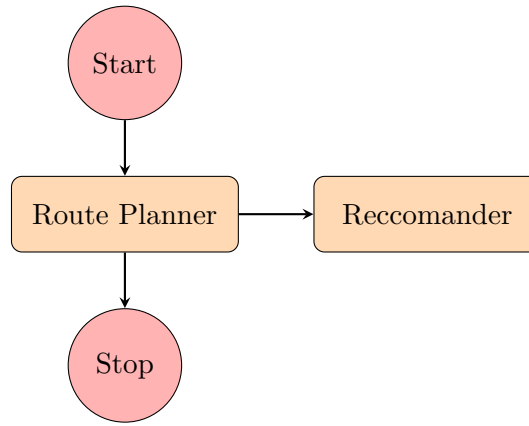


Figure 3.2: Second architecture of the system.

3.4 Technological choices

This section presents the reasons behind the choice of the technology used in the project implementation, which is discussed in Section 4.

- **Python:** Was chosen as the development language for its rich ecosystem for AI and machine learning applications. Secondly, being Python a really “simple language”, it is ideal for the scenarios in which fast prototyping capabilities are required, allowing for fast code drafting and development.
- **Pydantic-API³:** Was selected as the framework to realise the multi-agent architecture, mainly for its dependency injection feature that easily enables shared state management between the agents of the system. On top of that, being developed by Pydantic, there is the runtime validation of the data structures that maintain the conversational state and trip planning information.
- **GPT-4.1-mini:** Among the different models in the OpenAI family, the 4.1 family was selected for its balance between performance and cost. As an improvement from the previous family, the 4.1 models are trained to follow more closely the provided instruction, making it more sensitive to prompt, therefore making its performance more dependent on prompt quality. Lastly, among GPT-4.1, GPT-4.1-mini and GPT-4.1-nano, the mini model was chosen, since it offers a good balance between intelligence and latency.
- **External services:** To address routing and recommendation capabilities: three services based on OpenStreetMap (an open-source world map maintained by volunteers) that can be used as a REST API, where selected:
 - **Nominatim⁴:** As the geocoding service used to find places of the world using their names or addresses, to then obtain their geographical location (longitude and latitude).
 - **Brouter⁵:** As the cycling-specific routing service, it uses information such as the bike profile to influence the type of terrain used for the route.
 - **Overpass API⁶:** Enable for POI discovery, providing the recommendation capabilities for the trip personalisation.

- **Data-structures:** The data-structures design employs Pydantic models to ensure type safety across the conversational system. The `TripDescriptor` and `UserDescriptor` classes serve as the primary state management mechanisms, providing structured storage for trip planning parameters and user preferences, while enabling validation of user inputs against domain constraints.

These technological choices collectively provide the implementation foundation for the multi-agent conversational system described in Section 3.3, establishing the technical framework for the implementation discussion presented in Chapter 4.

4

System implementation

4.1 Final architecture

The system implements a single-agent architecture with tool-augmented capabilities, representing an explicit design choice that prioritises conversation continuity and state consistency over distributed processing.

Among the advantages offered by the chosen architecture, there is the centralization of the decision making process. A single route planning agent serves as the primary interface, making all decisions regarding information gathering, data validation, monitoring, and trip generation. This centralization ensures consistent behaviour and eliminates potential conflicts that could arise from multi-agent coordination (the same reasoning can be applied to the agent responsible for the route personalization).

Obviously, the architectural choice is not exempt from having flaws, indeed, with such an architecture, it is easier to overload the agent with too much responsibility and tools it can use, raising the level of confusion that the agent might develop. It is, in fact, a good practice to specialize the agents, making them address specific roles. However, even though from the architecture exposed and discussed in Section 3.3.2 there is a clear distinction between the agent charged for the basic route planning and segmentation and the agent responsible for the route personalization with POIs, in the current implementation, the second agent, since it was not the goal of this thesis to implement it, was developed as a set of tools made available to the first agent, and part of the prompt that instructs the first agent is dedicated to the realization of such a basic recommendation system.

4.1.1 Component architecture and dependencies

System entry point. The system entry point is defined in `main.py`, which serves as the orchestration layer, responsible for the dependency container initialization and launching the conversational agent:

```
1 def run_cycling_trip_agency():
2     """Main execution function for the director agent"""
3     deps = MyDeps(TripDescriptor(), UserDescriptor(), Recommendation())
4     route_planner.run_sync(deps=deps)
```

This design establishes a clear separation between system initialisation and agent execution, facilitating testing and modularity.

Agent definition. The primary agent is defined in `crew/route_planner_agent.py` using the Pydantic-API¹ framework. The agent configuration specifies:

- **Model selection:** GPT-4.1-mini, loaded from `crew/crew.yaml`, the agent configuration file, that stores the agent static prompt and LLM model name.
- **Dependency injection:** Typed dependency container, used to manage the shared state and data integrity.
- **Tool registration:** Agent instrumentation with a comprehensive set of tools aimed at the task fulfilment.
- **System prompt management:** Multi-modal system prompt injection, static system prompt and context-dependent dynamic prompt.

MyDeps container. The `MyDeps` container serves as the dependency injection mechanism, encapsulating both state management components:

```
1 @dataclass
2 class MyDeps:
3     trip: TripDescriptor
4     user: UserDescriptor
5     recommendation: Recommendation
```

This design guarantees type-safe dependency injection, ensuring that all agent operations have access to properly structured trip and user related information, while maintaining a clear separation of concerns. The `TripDescriptor`, `UserDescriptor`, and `Recommendation` classes all handle data validation and state persistence management:

- **TripDescriptor:** Manage trip-specific information, including bike type, locations, duration, and generated routes.
- **UserDescriptor:** Encompass user performance metrics and preferences for trip personalisation.
- **Recommendation:** Handle recommendations related information.

The descriptors above implement comprehensive validation logic, with the support of Pydantic models, and ensure data integrity throughout the conversational workflow.

4.2 Data structures

The data structures in the Cycling Trip Agency system are implemented using Pydantic `BaseModel` as the foundational framework, providing runtime type validation. This architectural choice ensures type safety, data integrity, and consistent validation across all system components.

¹Pydantic-AI: <https://ai.pydantic.dev/>

The data-structures serve as the central state management mechanism, maintaining conversation context, trip and user related information throughout the user interaction lifecycle. Each structure implements a comprehensive validation system that enforces domain-specific rules, while providing meaningful error feedback.

4.2.1 TripDescriptor implementation

The `TripDescriptor` class serves as the primary container for all trip-related information, implementing a validation and state management system, while holding the route planning logic.

Core structure.

```

1 class TripDescriptor(BaseModel):
2     """..."""
3     bike_type: str | None = None
4     places: list[Place] | None = None
5     duration: int | None = None
6     dates: list[date] | None = None
7     candidate_routes: list[list[list[float]]] | None = None
8     selected_route: int | None = None
9     stepped_route: list[list[list[float]]] | None = None
10    length: float | None = None
11    positive_height_difference: float | None = None
12
13    # ...

```

Some attributes that might need to be explained are `candidate_routes`, that holds four different route proposals based on the `bike_type` and places specified by the user, and `selected_route`, that holds the index of the route chosen by the user (among the ones proposed) based on the sequential route division in steps, `stepped_route`, based on performance metrics maintained by `UserDescriptor`.

Validation logic. The `TripDescriptor` implements a layered validation system, with private methods that enforce domain constraints.

```

1 # ...
2 def __set_bike_type(self, bike_type: str):
3     if not bike_type in ["road", "gravel", "mtb"]:
4         raise Exception(f"Error in TripDescriptor.__set_bike_type()\n"
5                         f"The given bike_type must be one of BikeType type\n"
6                         f"{bike_type} was provided")
7     self.bike_type = bike_type
8
9 def __set_places(self, places: list[str]):
10    if not len(places) > 1:
11        raise Exception(f"Error in TripDescriptor.__set_places()\n"
12                        f"The given places must contain at least 2 elements\n"
13                        f"{len(places)} were provided")
14
15    self.places = [Place(name=plc) for plc in places]
16

```

```

17 not_found = ""
18 for place in self.places:
19     if place.get_name() == "":
20         not_found += f"{place.get_users_name()}, "
21 if not_found != "":
22     raise Exception(f"Error in TripDescriptor.__set_places()\n"
23                     f"For the following places were not found: {not_found}")
24
25 # ...

```

This validation approach integrates automatic geocoding through the `Place` class, which performs real-time location resolution using the Nominatim² service, ensuring that all specified locations are valid and geocoded before trip planning proceeds.

Unified fill interface. The central `fill()` method provides a unified interface for updating trip information, while maintaining data consistency:

```

1 def fill(self, bike_type: None | str = None, places: None | list[str] = None,
2         duration: None | int = None, dates: None | list[str] = None,
3         selected_route: None | int = None):
4     """Fill the TripDescriptor with the given info
5     ...
6     """
7
8     if bike_type is not None:
9         try:
10             self.__set_bike_type(bike_type)
11         except Exception as e:
12             raise e
13
14     if places is not None:
15         try:
16             self.__set_places(places)
17         except Exception as e:
18             raise e
19
20     if duration is not None:
21         try:
22             self.__set_duration(duration)
23             self.__correct_eventual_inconsistency_between_dates_duration()
24         except Exception as e:
25             raise e
26
27     # ...

```

In the code above, there is an example of automatic consistency check, with the method `__correct_eventual_inconsistency_between_dates_duration()`, that is in charge of automatically adjusting eventual conflicts between trip duration and dates.

²Nominatim: <https://nominatim.org/>

Route planning integration. The `TripDescriptor` integrates route planning capabilities directly into the data structure.

```

1 def plan_candidate_routes(self):
2     """Plan 4 different routes that goes through the places provided"""
3     if self.places is None or len(self.places) < 2:
4         raise Exception("Error: places are not set")
5     if self.bike_type is None or self.bike_type not in ["road", "gravel", "mtb"]:
6         raise Exception("Error: bike_type is not set")
7
8     self.candidate_routes = []
9     for i in range(4):
10        route = self.__plan_route(i)
11        if len(route) > 0:
12            self.candidate_routes.append(route)

```

4.2.2 UserDescriptor implementation

The `UserDescriptor` class manages user-specific information through a compositional architecture, that separates the user performance metrics from its preferences specification.

Compositional structure.

```

1 class UserDescriptor(BaseModel):
2     performance: PerformanceDescriptor = PerformanceDescriptor()
3     preferences: PreferencesDescriptor = PreferencesDescriptor()

```

Performance management. The `PerformanceDescriptor` encapsulates user cycling performance metrics:

```

1 class PerformanceDescriptor(BaseModel):
2     kilometer_per_day: int = 0
3     difference_in_height_per_day: int = 0

```

This structure captures quantitative user capabilities that directly influence route segmentation.

Preference management. The `PreferenceDescriptor` class manages user interests for the trip personalisation:

```

1 class PreferencesDescriptor(BaseModel):
2     amenity: list[str] | None = None           # restaurants, cafes, shops
3     tourism: list[str] | None = None           # museums, galleries, attractions
4     natural: list[str] | None = None           # beaches, mountains, forests
5     historic: list[str] | None = None          # castles, monuments, sites
6     building: list[str] | None = None          # architecture, landmarks
7     water: list[str] | None = None             # lakes, rivers, waterfalls
8     leisure: list[str] | None = None           # parks, recreation areas
9     man_made: list[str] | None = None          # bridges, towers, structures

```

4.2.3 Tool structure

The system implements a tool-mediated integration pattern where agent tools serve as controlled interfaces to the data structures, providing error handling and context management.

Tool interface design. The integration tools act as an interface that wraps the data structure methods.

```

1 def fill_trip_description(ctx: RunContext[MyDeps], bike_type: None | str = None,
2                           places: None | list[str] = None, duration: None | int = None,
3                           dates: None | list[str] = None,
4                           selected_route: None | int = None) -> None | str:
5     """A tool to fill the trip description"""
6     try:
7         ctx.deps.trip.fill(bike_type, places, duration, dates, selected_route)
8     except Exception as e:
9         return str(e)

```

Error handling strategy. As shown in the code snippet above, the tools implement a graceful error handling logic that converts exceptions into error messages that are then passed to the agent. This mechanism allows the agent to handle the errors encountered during the execution.

4.3 Conversational agents

The conversational agent in the Cycling trip agency system is implemented using the Pydantic-API³ framework, which provides a robust foundation for building type safe and tool augmented language model agents.

4.3.1 Framework integration

```

1 import logfire
2 from pydantic_ai import Agent, RunContext, Tool
3
4 # Initialize comprehensive logging and instrumentation
5 logfire.configure()
6 logfire.instrument_pydantic_ai()
7 Agent.instrument_all()

```

This instrumentation provides complete visibility of the agent behaviour, enabling detailed tracking of the decision-making process, tool invocations, and conversation flow (useful capabilities for both the development and validation phases).

4.3.2 Configuration management

The system employs a YAML-based configuration approach that separates the agent behaviour specification from its implementation code.

³Pydantic-AI: <https://ai.pydantic.dev/>

```

1 with open("./crew/crew.yaml", "r") as file:
2     crew_info = yaml.safe_load(file)
3
4 route_planner = Agent(
5     model=crew_info["route_planner"]["llm"],
6     deps_type=MyDeps,
7     system_prompt=crew_info["route_planner"]["system_prompt"],
8     tools=[...]
9 )

```

This separation enables rapid experimentation of different prompt formulations and offers a centralised solution to system component configuration storage.

4.3.3 Tool integration

The agent incorporates a comprehensive suite of tools, each of which addresses a specific part of the trip planning process through the Tool object.

```

1 tools=[
2     Tool(say_to_the_user, takes_ctx=False, docstring_format="google", max_retries=3),
3     Tool(fill_trip_description, takes_ctx=True, docstring_format="google", max_retries
4         =3),
5     # Additional tools...
6 ]

```

Each Tool object has different configuration parameters:

- **takes_ctx**: Specify whether or not the tool requires access to the conversation context (in which the MyDeps container is contained).
- **docstring_format**: Ensure correct tool documentation parsing in the prompt.
- **max_retries**: Provides resilient error handling with automatic retry capabilities.

This configuration ensures that tool failures do not immediately terminate the conversation. Combined with the other error handling techniques introduced in Section 4.2.3, it allows for recovery through a retry mechanism.

4.4 Prompt engineering and desing

The system implements a multi-layered prompt architecture, that combines static information with context structure injection, enhanced with prompt engineering techniques.

The prompt architecture operates on two layers:

Static system prompt. It instructs the agent, defining its behaviour and extensively explaining the tasks it has to accomplish.

```

1 You are an expert cycling trip planning agent whose goal is to create personalized
  cycling routes that ...
2 # Reasoning patterns and rules
3
4 ## Patter
5
6 EVERY time you receive a message from the user, you MUST undergo this reasoning
  pattern:
7 ...

```

Structural context injection. It exposes the structure of the descriptors (TripDescriptor and UserDescriptor) that are in the execution context.

```

1 @route_planner.system_prompt()
2 def add_descriptors_structure_to_system_prompt(ctx: RunContext[MyDeps]) -> str:
3     return f"""The trip is described by the following class:\n{ctx.deps.trip.
  get_class_description()}
4 The user is described by the following class:\n{ctx.deps.user.get_class_description()}
  """

```

4.4.1 ReAct approach implementation

The system employs the ReAct (Reasoning and Act) approach, which improves agent performance through structured and interleaved reasoning and action phases. It provides explicit guidance for alternating between reasoning steps (analysis and planning) and action steps (tool execution).

```

1 EVERY time you receive a message from the user, you MUST undergo this reasoning
  pattern:
2
3 ### Thought
4     - User input analysis: [What the user just provided - be specific]
5     - Information extraction: [What are the information provided by the user ...]
6     ...
7 ### Action
8     - Perform the actions you have planned in the previous thought phase.
9
10 ### Reiterate the process
11     - Once received the outcome of the action you take, reiterate the process.

```

In addition to the ReAct prompting, detailed information about the information processing and reasoning approach of the agents was provided.

```

1 ## Critical Reasoning Rules
2
3 ### Always Reason About:
4     1. **Information Completeness**: What do I have vs. what do I need?
5     2. **Validation State**: Is the information logical and consistent?
6     ...

```

```

7 ### Never Skip Reasoning For:
8   - Unit conversions (show calculations)
9   - Conflict resolution (explain why you choose one option)
10  ...
11 ### Reasoning Quality Checks:
12   - **Specific**: "User provided distance" -> "User provided 30 miles = 48.3km daily distance"
13   - **Complete**: Analyze ALL aspects of user input, not just obvious ones
14  ...

```

4.4.2 Workflow deligation

In order to improve agent behaviour consistency with the expected workflow, the succession of steps involved in trip planning was explained in detail, highlighting the sub-steps needed to address the task, even suggesting the tools the user should use to accomplish the job, and explicitly stating the conditions for transitioning to the next phase.

```

1 # Workflow (steps involved in the planning)
2 ## 1. Information gathering
3 Goal: Collect essential information to plan the route.
4 ### Mandatory information
5 ...
6 ### Conversational strategy:
7 ...
8 ### Transition to the next step:
9 ...
10 ## 2. Candidate route selection
11 Goal: Generate multiple route options and help the user to choose the preferred one.
12 ...

```

4.4.3 Error handling

In order to improve the error handling capability, an agent was added in the prompt, a section dedicated to the explanation of how to approach error handling and tool failure.

```

1 # Error handling and data validation
2 ## Information consistency check
3 Goal: Understand the consistency of the information provided by the user and correct eventual errors.
4 ...
5 ## Tool failure handling
6 ...

```

4.5 Integration with external services

The Cycling Trip Agency system integrates three external services based on OpenStreetMap (OSM) data that provide comprehensive geospatial functionality. This integration strategy leverages open source

geospatial services that offer specialized capabilities for different trip planning aspects, from location resolution to cycling specific routing.

4.5.1 Geocoding services

Nominatim⁴ serves as the primary geocoding service, converting human-readable place names into geographical coordinates. This integration is essential for transforming user provided location names into the coordinate data required for the route planning.

Nominatim⁴ functions as a web-based REST API service that provides global geocoding capabilities using OSM data.

The system integrates Nominatim⁴ through `Place` class, which automatically performs geocoding during the object initialization.

```

1 Class Place(BaseModel):
2     # ...
3     def model_post_init(self, __context__=None) -> None:
4         self.__set_coordinates()
5
6     def __set_coordinates(self) -> None | str:
7         import requests
8
9         params = {...}
10        headers = {...}
11        url = f"https://nominatim.openstreetmap.org/search"
12
13        response = requests.get(url, params=params, headers=headers)
14        response.raise_for_status()
15        # ...
16    # ...

```

4.5.2 Routing services

Brouter⁵ provides cycling specific routing capabilities that consider cycling relevant constraints, such as the bike type that determines the terrain type for the trip. This service is essential for generating proper cycling routes between the geocoded locations.

Brouter⁵ operates as a locally hosted routing engine, run through a container, and it is queried by the `TripDescriptor` class. It offers the capability of generating routes through the specification of some parameters:

- **lonlats:** Coordinate pair, in longitude, latitude format, of the starting and ending point of the route.
- **profile:** That identifies the type of bike used for the trip, which influences the terrain type used for the trip.

⁴Nominatim: <https://nominatim.org/>

⁵Brouter: <https://brouter.de/brouter/>

- **alternativeidx**: It is an index used to obtain different route variants from Brouter⁶. It is used by the `TripDescriptor` to obtain up to four candidate routes that go through the specified locations.
- **format**: This allows you to specify the expected response format.

```

1 class TripDescriptor(BaseModel):
2     # ...
3     def __plan_route(self, idx: int) -> list[list[float]]:
4         """Get a route that goes through the places provided"""
5         bike_profile = self.bike_type
6         if self.bike_type == "road":
7             bike_profile = "fastbike"
8
9         locations_coordinates = [place.get_coordinates() for place in self.places]
10        route = []
11        for i in range(1, len(locations_coordinates)):
12            lon_lat_string = f"{locations_coordinates[i-1]\cite{bib:1}},..."
13            url = f"http://localhost:17777/brouter?lonlats={lon_lat_string}&profile={
bike_profile}&alternativeidx={idx}&format=geojson"
14            response = requests.get(url)
15            response.raise_for_status()
16
17            route.extend(response.json()["features"][0]["geometry"]["coordinates"])
18
19        return route
20    # ...

```

Since Brouter⁶ can be used to obtain a route from a point A to a point B without being able to specify intermediate points, the route generation process follows a segment based approach, constructing the route connecting the different segments.

4.5.3 Recommendation service

The Overpass API⁷ provides POI discovery capabilities, used for trip personalisation. It was used to implement the dummy recommendation system; even though that component was not addressed in this chapter, it is worth mentioning how it was used, since it can be used as the foundation of a complete recommendation system for future development.

Overpass API⁷ is a web-based REST API service that allows to query for POIs based on the POI category, such as amenity, natural, and proximity to a certain location, specifying a location and a radius.

```

1 class Recommendation(BaseModel):
2     # ...
3     @classmethod
4     def __query_overpass(cls, query: str, max_retries: int = 3) -> requests.Response:
5         url = "https://overpass-api.de/api/interpreter"

```

⁶Brouter: <https://brouter.de/brouter/>

⁷Overpass API: <https://overpass-api.de/>

```
6
7     for i in range(max_retries):
8         try:
9             response = requests.post(url, data=query, headers={'Content-Type': '
application/x-www-form-urlencoded; charset=UTF-8'})
10            response.raise_for_status()
11            return response
12        except Exception as e:
13            if i != max_retries - 1:
14                time.sleep(2 ** i + random.uniform(0, 1))
15                continue
16            raise e
17 # ...
```

5

Testing and validation

5.1 Testing methodologies

The testing strategy adopted during the development of the Cycling Trip Agency project was designed to ensure both component reliability and system integration quality. Given the multi-layered architecture, encompassing data-structures, external service integrations and conversational agents, a hybrid testing approach was adopted, with black-box testing for individual components and bottom-up integration testing for system validation.

5.1.1 Black-box component testing

Black-box testing was selected as the primary approach for testing the individual components, focusing on behavioural validation rather than internal implementation detail. This methodology proved to be particularly effective for several reasons:

- **Data structure validation:** The core data structures (`TripDescriptor`, `UserDescriptor`) serve as the foundation of the system state management. Black-box testing enabled the validation of their public interface:
 - Input validation constraints (e.g. bike type existence, minimum places requirements).
 - Error handling provides meaningful feedback on the violated constraint.
- **Tool behavioural verification:** Tool behavioural verification: The system tools, such as geocoding, routing and data filler, required testing focused on input-output correctness.
 - Validation of the data transformation between different tools and services.
 - Error handling for external services failures.
- **Interface contract testing:** Testing the tools based on their signature and expected behaviours, the black-box approach guaranteed that each component fulfils its contractual obligations.

5.1.2 Bottom-up integration testing

Bottom-up integration testing was chosen over its alternatives, such as the top-down approach, based on its characteristics that made it more suitable for the needs of the development workflow adopted:

- **Minimal stub requirements:** Unlike top-down testing, the bottom-up approach eliminates the need for complex stubs. Given the architecture, where higher-level agents rely on fully functional data-structures and tools, this approach required only production equivalent code to mimic realistic testing environment.
- **Progressive validation:** This methodology ensured that higher level components were built upon, tested and verified lower level components. This testing approach ensured:
 - External services integration worked correctly before being used by the tools and data structure.
 - Data-structures were fully functional before tools and agent integration.
- **Early detection of component issues:** By testing the components in isolation first, integration testing could be clearly attributed to interface mismatches, rather than component level bugs, reducing debugging complexity.

5.1.3 Approach trade-offs

While this testing approach proved to have significant advantages, like being code efficient (compared to its counterpart), some limitations it has are worth mentioning:

- **Late architecture validation:** The overall system architecture and public interfaces were tested only after complete implementation. This could possibly result in a redesign phase, if architectural inconsistencies are discovered during full system testing.
- **Limited top-level behaviour testing:** Early stage testing focuses on component correctness rather than end-to-end user experience validation, potentially missing workflow-level issues until later development stages.

5.2 Agent behaviour testing

The framework chosen to support the agent behavioural testing phase was Logfire¹, an observability and monitoring library. Logfire¹ was selected due to its native integration with Pydantic-API² and its comprehensive tracing capabilities for conversational agent systems.

5.2.1 Framework integration

The framework initialization implemented in the main application entry point establishes, by itself, a comprehensive monitoring coverage:

¹Logfire: <https://pydantic.dev/logfire>

²Pydantic-AI: <https://ai.pydantic.dev/>

Listing 5.1: LogFire initialization code snippet

```

1 import logfire
2 from pydantic_ai import Agent
3
4 logfire.configure(send_to_logfire="if-token-present")
5 logfire.instrument_pydantic_ai()
6 logfire.instrument_requests()
7 Agent.instrument_all()

```

This configuration enables automatic instrumentation of all Pydantic-API³ agents and their interactions, providing detailed execution traces without manual instrumentation for each component. Beyond the automatic instrumentation, manual logging was placed in critical system points, such as YAML configuration parsing and agent creation.

5.2.2 Monitoring capabilities

The Logfire⁴ framework provided a comprehensive visibility of multiple aspects of the system execution, which proved to be essential for both development and testing phases:

- **Prompt monitoring and analysis:** One of the most crucial capabilities was the tracing of the prompt generation and modification. Enabling the capture of static and dynamic system prompts.
- **Agent decision tracking:** The framework provided detailed insights on the agent's reasoning process. Enabling the observation of reasoning traces, monitoring of the parameter extraction from the user input and tool responses (allowing tracking and state monitoring of the call to the external services).

5.2.3 Traces analysis methodology

The foundation of behaviour tracking was built upon a systematic analysis of Logfire⁴ execution traces, which provided chronological records of agent interactions with the user. This enabled the analysis of different aspects during the execution:

- **Conversational flow mapping:** Each test scenario generated a complete conversational trace that was analysed to understand the agent decision pathways. The tracking methodology focus on identifying the decision points where the agent chose between different response strategies, tool selection or information extraction approaches. This analysis revealed patterns in the agent reasoning, where it deviated from the expected behaviour.
- **Tool interaction analysis:** The detailed tracking of the agent–tool interactions provided insights into the decision quality and execution efficiency. The methodology analysed the tool selection appropriateness to the context, parameter extraction accuracy, tool call sequencing, and error handling recovery (such as an inconsistency between the information provided by the user or a tool failure).

³Pydantic-AI: <https://ai.pydantic.dev/>

⁴Logfire: <https://pydantic.dev/logfire>

- **Comparative analysis:** An important aspect of behaviour tracking involves comparing agent performance across different prompts. The tracking methodology exposed above enabled a direct comparison of the agent behaviour between the prompt evolutions subjected to the evaluation, making it easier to understand if the changes made to the previous prompt iteration have led to the desired agent behaviour changes.

5.3 Experimental validation design

The experimental validation of cycling trip agency system focuses on evaluating the agent performance across different prompt engineering approaches.

5.3.1 Experimental objectives

The primary goal was to quantify the impact of the prompt engineering techniques on the conversational agent's effectiveness in extracting structured information from natural language inputs. The experiment aimed to:

- **Performance qualification:** Measure the accuracy and completeness of information extraction across eight categories: bike type, trip places, trip duration, travel dates, daily distance and elevation capacity, user preferences, and user preferences notes (additional notes about the user preferences).
- **Prompt Engineering Effectiveness:** Evaluate the impact of the ReAct methodology on the system prompts, comparing to a baseline structured prompt.

5.3.2 Experimental subject

Two distinct prompt formulations were developed and tested:

- **Prompt 1 (Baseline):** A structured prompt, encompassing four main sections:
 - *Behaviour section:* Basic interaction and behavioural rules that guide agent responses.
 - *Workflow section:* Sequential steps for trip planning with completion criteria and tool usage guidance.
 - *Error handling and data validation:* Basic strategies for managing user input errors and data contradictions.
 - *Tool usage guidelines:* Brief description of the purpose of the different tools.
- **Prompt 2 (Enhanced ReAct):** A prompt with the same structural foundation as Prompt 1, but with the following aspects enhanced:
 - *Interleaved reasoning and action framework:* Explicit guidance for alternating between reasoning steps (planning and analysis) and action steps (tool execution).
 - *Structured decision-making process:* Enhanced guidance for a systematic evaluation of available information before proceeding with the subsequent steps.

- *Smaller tweaks*: To improve error handling, information, consistency checks and tool failure.

Information Extraction Performance: Measured through binary success metrics across eight information categories.

5.3.3 Test scenarios development

The experimental design encompasses thirty test scenarios to provide comprehensive coverage of message types:

- **Realistic use case scenarios:**
 - *Comprehensive messages*: Scenarios where the needed information were provided at once.
 - *Partial information*: Scenarios where a subset of the needed information were provided.
- **Edge case boundary testing:**
 - *Domain boundary violations*: Testing agent responses to non-cycling related requests.
 - *Ambiguous input handling*: Scenarios with unclear or contradictory information.
 - *Error recovery testing*: Cases with malformed inputs of invalid data.

5.4 Agent experimental evaluation

The evaluation of the agent response required an assessment methodology that could systematically analyse both the quality and completeness of information extraction across the two prompt formulations.

5.4.1 Evaluation framework architecture

Information Extraction Assessment: The primary evaluation metric focused on the agent ability to successfully identify and extract structured information from natural language inputs, from each of the eight information categories (bike type, places, duration, dates, daily distance and elevation, preferences, and preferences notes).

Prompt performance comparison: The secondary evaluation metric involved the comparison of the results of the agent instructed on the second prompt against the results obtained with the first prompt.

5.4.2 Binary classification methodology

The core evaluation approach employed binary classification for each information category, providing, clear quantifiable metrics for comparative analysis:

- **Success criteria**: For each information category, specific success criteria were established based on their respective constraints:
 - *Bike type*: Successful categorisation into road, gravel or mtb.
 - *Places*: Accurate extraction with successful geocoding of the locations, with minimum quantity constraints.

Prompt 1 captured information								
	bike_type	places	duration	dates	km_daily	elv._daily	pref.	pref._note
1	✓	✓	✓					
2	✓				✓	✓		
3	✓	✓						
4		✓						
5		✗						
6								
7	✓	✓			✓	✓		
8	✗	✓						
9		✓			✓	✓		
10		✓	✓					
11		✓			✗			
12	✗	✗	✓					
13		✓		✗				
14		✓				✗		
15		✓						✓
16		✓						✓
17		✗					o	
18	✓	✓			✓	✓		
19	✓	✓						
20	✓	✓						
21	✓	✓			✓	✓		
22		✓			✗			
23	✓	✓						
24	✓	✓			✓	✓		
25	✗				✗			
26		?						
27					✓		o	✓
28	✓	✓						
29	✓	✓						
30		✓		✓				

Table 5.1: The results to the tests on the agent instructed with Prompt 1.

- *Duration / Dates*: Correct parsing of temporal information with appropriate formatting.
- *Performance Metrics*: Accurate extraction and validation of numerical performance data.
- *Preferences*: Successful identification and categorisation of user preferences for trip personalisation.

- **Classification protocol**: Each test scenario response was evaluated against the success criteria and mapped to a binary outcome: ✓ for successful extraction, ? for partial information, ✗ for hallucinated information, o for missing information.

5.5 Experimental results

Experimental validation revealed a significant improvement in the agent’s performance when instructed with the second prompt incorporating the enhanced ReAct approach. Evaluation across thirty test scenarios demonstrated measurable improvements in extraction accuracy.

Expected result	Unexpected result
63%	37%

Table 5.2: Overall performance of the agent instructed with Prompt 1.

5.5.1 Overall performance comparison

The most significant result was the noticeable improvement in overall system performance between the two prompts, which scored as follows:

- **Prompt 1 (baseline):** As the Table 5.2, the structured prompt achieved a success rate of 63% across all test scenarios. However, while it demonstrated functional capabilities, it was found to have substantial limitations.
- **Prompt 2 (Enhanced ReAct):** As the Table 5.4, the enhanced ReAct prompt achieved an 80% success rate, representing a relative improvement of almost 27% over the baseline prompt. This improvement in performance demonstrates the effectiveness of implementing structured reasoning-action methodology in conversational agent design.

The 17% improvement in correctness of the second prompt over the first suggests that the ReAct approach addresses a fundamental limitation in the agent’s decision-making process.

5.5.2 Information quality analysis

A detailed analysis of information quality revealed substantial differences in agent behaviour, beyond the overall success rates.

- **Prompt 1:** As table 5.1 shows, out of all the extracted information, 81% was correct, 17% consisted of hallucinations, and the remaining 2% was misinterpreted, while the 4% of the information provided to the agent is missing.
- **Prompt 2:** As table 5.3 shows, out of all the extracted information, 87% was correct, while the remaining 13% was hallucinatory.

The ReAct methodology has demonstrated a relative reduction of 23.5% in hallucination rate (from 17% to 13%) over the first prompt, indicating improved information reliability and reduced generation of unsupported claims. Furthermore, Prompt 2 has completely eliminated the extraction of factually incorrect information, suggesting that structured reasoning improves validation and consistency checks.

5.5.3 Message complexity analysis

The experimental test suite was organised into four distinct categories of increasing complexity, enabling a detailed analysis of where the ReAct approach provided the most significant benefits.

- **Base messages (13 scenarios):** Those are straightforward query, with standard formatting and clear information. This category of tests have seen a 25.2% relative improvement.
 - *Prompt 1:* 8 successful extraction (61.5% success rate)

Prompt 2 captured information								
	bike_type	places	duration	dates	km_daily	elv._daily	pref.	pref._note
1	✓	✓	✓					
2	✓				✓	✓		
3	✓	✓						
4		✓						
5		?						
6								
7	✓	✓			✓	✓		
8		✓						
9		✓			✓	✓		
10		✓	✓					
11		✓						
12		?	✓					
13		✓		✗				
14		✓				✗		
15		✓						
16		✓						
17							✓	
18	✓	✓			✓	✓		
19	✓	✓						
20	✓	✓						
21	✓	✓			✓	✓		
22		✓			✗			
23	✓	✓						
24	✓	✓			✓	✓		
25	✗	✗			✗			
26		✓						
27					✓		✓	
28	✓	✓						
29	✓	✓						
30		✓		✓				

Table 5.3: The results to the tests on the agent instructed with Prompt 2.

- *Prompt 2*: 10 successful extraction (77% success rate)
- **Inference required messages (5 scenarios)**: Represents messages requiring logical deduction or implicit information extraction. This category of tests have registered a 25% relative improvement.
 - *Prompt 1*: 4 successful extraction (80% success rate)
 - *Prompt 2*: 5 successful extraction (100% success rate)
- **Ambiguous messages (7 scenarios)**: Are scenarios with unclear or potentially contradictory information. This category registered a relative increase of the 25.2%.

Expected result	Unexpected result
80%	20%

Table 5.4: Overall performance of the agent instructed with Prompt 2.

- *Prompt 1*: 4 successful extraction (57% success rate)
- *Prompt 2*: 5 successful extraction (71.4% success rate)
- **Edge case messages (5 scenarios)**: Encompasses out-of-domain requests, malformed input or boundary-testing scenarios. This category of test have registered a 33.3% relative improvement.
 - *Prompt 1*: 3 successful extraction (60% success rate)
 - *Prompt 2*: 4 successful extraction (80% success rate)

5.6 Conclusions

Experimental validation provides conclusive evidence of the effectiveness of advanced prompt engineering techniques in the development of conversational agents. The systematic performance improvements suggest that the ReAct methodology offers benefits beyond simple accuracy metrics, providing fundamental behavioural enhancements.

- **Broader enhancement pattern**: The performance analysis reveals that the ReAct approach provides consistent improvements across all complexity categories. This broader benefit pattern demonstrates that structured reasoning provides enhancement despite not being category-specific optimisations.
- **Consistent improvement**: This indicates that the React methodology delivers systematic rather than sporadic benefits. This consistency demonstrates the reliability of the approach when dealing with a variety of conversational challenges, ranging from simple queries to edge case scenarios.
- **Thoughtful action**: Enhancing the prompt by leveraging extensive thinking on the current state, combined with the systematic processing of information extracted from user input, enables a more thoughtful action phase. This structured approach significantly improves the overall performance by reducing errors committed by the agent (as demonstrated by the reduction in hallucinations and the absence of un-extracted information).
- **Critical quality improvement**: The main benefits of the ReAct enhancement are the reduction in agent hallucinations (from 17% to 13%) and the complete retrieval of information (from 96% to 100%).

Even though these results may not appear dramatic at first glance, it should be noted that only one technique was used. Therefore, the performance on these topics can be improved by incorporating other thoughtful, action-oriented techniques into the prompts to ensure more consistent and error-free execution.

6

Discussion and limitations

The system successfully addresses the core academic objectives established in Chapter 1. However, when evaluated against the original objective, it was clear that there were also areas that require further development.

6.1 Evaluation of the developed prototype

6.1.1 Technical achievements

From a technical perspective, the prototype successfully realises several key components of the envisioned architecture. The agentic framework, built with Pydantic-API¹, demonstrates effective dependency injection and shared state management capabilities between the system components. The integration with external services (Nominatim² for geocoding, Brouter³ for routing, and Overpass API⁴ for POI recommendation) create a functional foundation for the trip planning capabilities. Whereas the data structures (`TripDescriptor` and `UserDescription`) provide data validation for maintaining data integrity through the conversational workflow.

The prompt engineering results highlighted the benefits drawn by the system from the ReAct methodology, not only improving the overall performance, but also reducing hallucination rate from 17% to 13%, demonstrating enhancement reliability in information processing and decision-making.

6.1.2 Prototype maturity assessment

The current implementation of Cycling Trip Agency can be categorised as an advanced proof-of-concept, transitioning toward a minimum viable product (MVP). While the system successfully demonstrates the core functionality of conversational trip planning and route generation, it lacks the comprehensive feature set and robustness required by a production ready software. The developed prototype effectively validates the feasibility of such a system, and provides a foundation for further development, requiring significant expansion to achieve the full version outlined in Section 3.2.

¹Pydantic-AI: <https://ai.pydantic.dev/>

²Nominatim: <https://nominatim.org/>

³Brouter: <https://brouter.de/brouter/>

⁴Overpass API: <https://overpass-api.de/>

The system successfully handles the initial phase of the trip planning workflow (information gathering and basic route generation), which aligns with the writer objectives underlined in Section 1.2.3. However, the broader academic objectives, exposed in Section 1.2.2, of providing a fully personalized cycling trip experience, including POIs integration and route refinement, remain partially fulfilled due to the limited implementation aims for the project.

6.2 Current limitations

As mentioned in the previous section, Section 6.1.2, the system is not production ready. This section will therefore explain the limitations identified in the system, which have been categorised as either technical or functional.

6.2.1 Technical limitation

Technical limitations are problems related to how the system works internally, such as implementation, agent behaviour or architecture.

- **Completeness of the components:** As underlined in Section 6.1.2, even though a part of the system is working, there are some components that miss a real or complete implementation. An important component that falls in that category, and must be developed in order for the system to be considered an MVP, is the Recommendation Agent. Once developed, the system will be able to provide a minimal but complete personalised cycling route planning experience.
- **Subtle requests:** More critically, the agent struggles with subtle, domain-adjacent requests. For example, when the user requests trips “from point A to point B with beautiful scenery to drive through”, the system attempts to fulfil the request without asking the user for clarification. That case represents a problem, since the agent uses cycling-specific routing algorithms that prioritize cyclable paths, that may not be suitable for automotive travelling.
- **Truly performance-fit route:** As up to now, the system uses user performance metrics (daily distance capacity and elevation tolerance) to divide the route into manageable steps. The problem lies in the fact that the approach used does not account for the dynamic impact of the terrain variation that can significantly impact the user actual performance. The slope of a part of the route affects the cyclist actual capabilities (a rider that can comfortably handle 80km per day on a flat surface might struggle to do the same amount of kilometres with climbs along the way). Despite the effect of slope on reducing the effective performance of the user, an even worse scenario is represented by the possibility for the user to encounter slopes too high for its level.

6.2.2 Functional limitation

Functional limitations refer to problems related to what the system can offer the user, such as user needs or missing features.

- **Lack of integration:** The system lacks the integration with real-time information, such as weather conditions and seasonal route accessibility. Cyclists planning a multi-day trip might

require this dynamic information to make a more informed decision about the trip timing and eventual route modifications.

6.3 Further development

As has emerged in the above section about system limitations (Section 6.2), there quite a few points where the system has to be improved. However, following are some proposals for future development that will make the system an MVP (and maybe a bit more):

- **Recommendation:** The development of a comprehensive recommendation agent represents the highest priority for achieving the MVP status. This component should implement POI discovery that goes beyond basic geospatial queries, providing suggestions not only based on user preferences, but also trip characteristics and seasonal considerations. Furthermore, it should encompass learning mechanisms that evolve suggestions based on the interaction pattern and feedback from the user.
- **Reasoning enhancement:** The prompt should be refined, adding a more structured approach for the user query breakdown and analysis, and multi-step reasoning. All of that to further lower the hallucinations and higher the understanding of the user request.
- **Domain boundary enforcement:** While the system demonstrates some domain awareness by referencing the trip planning procedure even when addressing unrelated queries, it should fundamentally reject non-cycling requests rather than attempting to answer them.
- **Implementation of a GUI:** The development of a comprehensive graphical user interface (GUI) represents an essential step toward usability. While the majority of the information handled by the system (such as bike type, locations and preferences) has a textual representation that can be understood by humans (such as the address or name of a place or building, or the name of the type of bike), other information, crucial for the trip planning scope, are hard to comprehend when displayed in their raw form on a terminal (the route is a succession of geographical points, and a geographical point is just a set of floating point numbers), hence there is the need to better display some data (maybe on a map), in order to make them understandable by the user.

7

Conclusion

The prototype developed through this thesis work provides a foundation for future development, while underlining the importance of structured reasoning in conversational agent design. The systematic application of the ReAct framework in a domain-specific context provides valuable insights on how structured reasoning can improve the conversational agent performance, achieving an 80% success rate compared to the 63% of the baseline prompt, and reducing agent hallucinations from 17% to 13%, raising the system reliability. Those results suggest that a further refinement of the prompt engineering techniques used for agent instructing can yield an even more significant enhancement on the system performance and reliability.

From a technical perspective, the prototype successfully implements the key components of the envisioned architecture, exposed throughout Chapter Requirements Analysis and Design. The agentic framework, built with Pydantic-API¹, shows effective dependency injection and shared state management capabilities between the different system components. While the integration with external services, Nominatim², Brouter³ and Overpass API⁴, created the functional foundation for trip planning capabilities. Lastly, data validation, to guarantee data integrity throughout the whole conversational workflow, was provided by the data-structures, **TripDescriptor** and **UserDescriptor**.

This thesis presents the design and implementation of an intelligent conversational system for personalised cycling trip planning. The research successfully demonstrated the feasibility of applying LLMs and conversational AI to create a more user-friendly and guided approach to cycling trip planning. The prototype validates the potential for AI-powered systems to transform how users interact with trip planning tools, moving from manual research and planning to a more guided, conversational experience. This shift can significantly lower the entry barrier to cycle tourism, while improving the quality and personalisation of trip planning.

¹Pydantic-AI: <https://ai.pydantic.dev/>

²Nominatim: <https://nominatim.org/>

³Brouter: <https://brouter.de/brouter/>

⁴Overpass API: <https://overpass-api.de/>

Bibliography

- [1] Legambiente, "Cicloturismo, "Viaggiare con la Bici 2025" • Legambiente", Legambiente. Accessed: Aug. 30, 2025. [Online]. Available: <https://www.legambiente.it/comunicati-stampa/cicloturismo-viaggiare-con-la-bici-2025/>
- [2] A.-K. Reibold and O. Malak, "Fabian Küster, Director of Advocacy and EU Affairs, f.kuester@ecf.com".
- [3] S. Bondarenko, O. Kalaman, and O. Danilova, "DIGITAL TRANSFORMATION OF THE TOURISM BUSINESS IN AN OPEN ECONOMY: A COMPREHENSIVE APPROACH TO TRAVEL PLANNING AND PARTNERSHIPS", SDELI, no. 4, Apr. 2025, doi: 10.70651/3083-6018/2025.4.17.
- [4] "What Is NLP (Natural Language Processing)? — IBM". Accessed: Aug. 31, 2025. [Online]. Available: <https://www.ibm.com/think/topics/natural-language-processing>
- [5] "Introduction to Large Language Models — Machine Learning — Google for Developers". Accessed: Aug. 31, 2025. [Online]. Available: <https://developers.google.com/machine-learning/resources/intro-llms>
- [6] "What are Large Language Models? — NVIDIA Glossary", NVIDIA. Accessed: Aug. 31, 2025. [Online]. Available: <https://www.nvidia.com/en-us/glossary/large-language-models/>
- [7] 'Agents'. Accessed: Sept. 26, 2025. [Online]. Available: <https://www.kaggle.com/whitepaper-agents>.
- [8] "Building Effective AI Agents — Anthropic". Accessed: Mar. 19, 2025. [Online]. Available: <https://www.anthropic.com/engineering/building-effective-agents>
- [9] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models", Mar. 10, 2023, arXiv: arXiv:2210.03629. doi: 10.48550/arXiv.2210.03629.
- [10] X. Wang and D. Zhou, "Chain-of-Thought Reasoning Without Prompting", May 23, 2024, arXiv: arXiv:2402.10200. doi: 10.48550/arXiv.2402.10200.
- [11] "IA conversazionale", Google Cloud. Accessed: Sept. 05, 2025. [Online]. Available: <https://cloud.google.com/conversational-ai>
- [12] "What is Conversational AI? — IBM". Accessed: Sept. 06, 2025. [Online]. Available: <https://www.ibm.com/think/topics/conversational-ai>

- [13] 'Bicycle touring', Wikipedia. Sept. 09, 2025. Accessed: Sept. 26, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Bicycle_touring&oldid=1310498587